Kortex

Nube segura de documentación

Wiki + ofimática + almacén seguro con permisos centralizados, CLI y MCP propios

Actividad 2 — Ingeniería del software (Tema 2) · Entornos de Desarrollo

David Cabrera González

Índice

- 1. Idea: qué es Kortex y qué problema resuelve
- 2. Justificación con conceptos del Tema 2
- 3. Documentación inicial: descripción, objetivos, roles y plataformas
- 4. Modelo de desarrollo: Scrum + espiral y roadmap de sprints
- 5. Requisitos funcionales y no funcionales
- 6. Claves: seguridad centralizada, carpeta-Nido y Fast Access
- 7. Fases aplicadas, casos de uso y conclusión

¿Qué es Kortex?

- Plataforma todo-en-uno que unifica tres cosas que hoy van por separado:
  - Wiki colaborativa (tipo Outline)
  - Suite ofimática completa: texto, cálculo, presentaciones y PDF (tipo ONLYOFFICE)
  - Almacén de archivos seguro con usuarios y permisos centralizados (tipo Samba)
- Toda la funcionalidad se expone por la web, por una CLI propia (kortex) y por un servidor MCP para IA.
- Orientada a empresas, con uso no comercial gratuito (estudiantes, ONG, proyectos personales).

La unidad no es una página: es una carpeta real; dentro conviven archivos y una wiki.

El problema que resuelve

- ✦ Fragmentación: el conocimiento vive en la wiki, los documentos en la ofimática y los archivos en el almacenamiento.
- ✦ Se repiten tareas manuales: actas, informes y plantillas hechos a mano.
- ✦ Los agentes de IA no tienen una vía estándar para leer o escribir documentación.
- ✦ Todo debe ser trazable y buscable para poder mantenerlo con los años.
- ✦ Se pierde conocimiento cuando alguien deja la empresa.

Solución: una sola carpeta y una sola identidad para todo el conocimiento del proyecto.

Justificación (conceptos del Tema 2)

1 · Ingeniería del software

- El tema: el software es "más que código" y documentar es imprescindible para mantenerlo.
- Kortex da soporte a la documentación y al mantenimiento.

2 · Fases del ciclo de vida

- Requisitos → análisis → diseño → implementación → pruebas → documentación → explotación → mantenimiento.
- El proyecto se planifica con estas fases.

3 · Metodologías

- Se descarta cascada y se propone Scrum (ágil) con un ciclo en espiral de validación.
- Caso práctico 2: adelantarse al competidor con sprints.

4 · Roles

- Analista de sistema, diseñador, analista programador, programador y arquitecto.
- Reflejados en el equipo que construye Kortex.

Documentación inicial

Nombre

- Kortex: Donde se almacena y conecta el conocimiento.
- CLI: kortex · Conector de IA: kortex-mcp.
- Espacios wiki internos: Nidos.

Descripción breve

- Wiki + edición ofimática + nube segura con identidad y permisos centralizados.
- El contenido vive en carpetas reales; dentro puede haber un Nido (wiki .md).

Objetivos

- 1. Unificar herramientas 2. Automatizar 3. Integrar IA
- 4. Trazabilidad y mantenimiento 5. Seguridad y control de acceso
- 6. Portabilidad (.md, Git) 7. Rendimiento sin perder seguridad (Fast Access)

Usuarios principales (roles) — ejemplo

Rol	Permisos	Tipo de usuario
Administradores	Usuarios, grupos, permisos, cuotas, despliegue (control total)	Gestión y sistemas
Editores o redactores	Crea y edita páginas, documentos y Nidos	Empleado normal
Lectores o Invitados	Consulta solo lectura (enlace con caducidad)	Cliente o terceros
Integradores	Conecta otros sistemas con el programa con la CLI y el MCP	Perfil técnico
Agentes IA	Permisos heredados del usuario	Automatización

Plataformas

Web y escritorio

- Web (navegador): Portabilidad y acceso inmediato.
- Escritorio: CLI multiplataforma y app ligera.

Móvil

- App básica para edición rápida (iOS y Android).

Servidor

- Docker/Kubernetes, en nube o local.
- API REST + MCP + SMB/WebDAV + caché RAM/WAL.
- La carpeta puede montarse como unidad de red.

Modelo de desarrollo: Scrum + espiral

- ¿Mejor que cascada? El sector cambia rápido: cascada obliga a publicar tarde y arriesga construir algo no pedido.
- Scrum (ágil): sprints de 2 semanas; cada uno viene con una nueva funcionalidad.
- Antes del primer sprint, un ciclo en espiral valida que carpeta + wiki + editor + permisos + API conviven.
- Referencia del tema: caso práctico 2 — con lo ágil se llega antes al mercado que un competidor en cascada.

Roadmap de sprints

Sprint	Entrega
0 · Espiral	Prototipo: Carpeta con wiki, editor básico, permisos y API
1	Identidad y permisos centralizados (LDAP / AD / SSO)
2	Almacén de archivos y montaje SMB/WebDAV (nube segura)
3	Wiki y Nidos (páginas .md anidadas en carpetas)
4	Editor ofimático con co-edición en tiempo real
5	CLI kortex
6	Servidor MCP para agentes de IA
7	Fast Access (RAM + diario/WAL + durabilidad)

Requisitos funcionales (qué hace) — 21

- Wiki jerárquica (colecciones y subpáginas) y editor ofimático con co-edición en vivo (DOCX, XLSX, PPTX, PDF).
- Búsqueda de texto completo que indexa tanto los archivos de la carpeta como las páginas de los Nidos.
- CLI kortex (crear, leer, actualizar y publicar desde la terminal) y servidor MCP para que los agentes de IA hagan lo mismo.
- Permisos por rol y ACL heredadas (carpeta → Nido → página), más historial de versiones con restauración.
- Identidad centralizada (LDAP / AD / Kerberos): una sola cuenta sirve para SMB, web y wiki.
- Montaje de la carpeta como unidad de red (SMB/CIFS/WebDAV): aparece como un disco más.
- Nidos: wiki de archivos .md anidada dentro de una carpeta real, editable desde disco o desde la web.
- Fast Access: carpetas en RAM con política de durabilidad y cuotas, con desalojo automático.
- Diagramas integrados tipo draw.io: se crean y editan dentro de Kortex y se guardan como un archivo del proyecto.
- Editor de HTML/CSS con visor en vivo: código a un lado y resultado al otro, con autoguardado que refresca la vista.

Un requisito funcional describe un comportamiento: qué hace el sistema ante una entrada o acción.

Requisitos no funcionales (cómo) — 12

- Rendimiento: La búsqueda y modificación es inmediata con Fast Access
- Escalabilidad: Horizontal con kubernetes.
- Seguridad: Cifrado en reposo y en tránsito + aislamiento por organización.
- Disponibilidad: Respuesta rápida y edición conjunta.
- Usabilidad y accesibilidad: español/inglés, WCAG 2.1 AA.
- Portabilidad: Docker/Kubernetes o en local, sin dependencia de un cloud.
- Interoperabilidad SMB2/SMB3 y conformidad con AD/Kerberos.
- Privacidad: Sin salida de datos.

Un requisito no funcional describe cómo debe comportarse: calidad, rendimiento, seguridad…

Seguridad e identidad centralizada (tipo Samba/AD)

- Directorio único (LDAP / Active Directory / Kerberos): Un registro por usuario, vale para SMB, web y wiki.
- Permisos heredados (ACL): carpeta → Nido → página, con excepciones configurables.
- Doble capa: ACL y RBAC.
- Montaje como unidad de red: El usuario ve su nube segura en el explorador de archivos.
- Endurecimiento: MFA, tokens con caducidad, cifrado en reposo y en tránsito.
- Control: Cuotas, enlaces caducables y auditoría de accesos (quién, qué, cuándo).
- El agente de IA hereda los permisos del usuario que lo invoca: puede hacer lo mismo que ese usuario, y nada más.

Resultado: una nube privada segura y, encima, la plataforma de edición y de notas.

El modelo carpeta-Nido (wiki dentro de una carpeta)

/proyectos/mi-proyecto/

```
├── program.java          # código
├── documentacion.docx    # ofimática
├── presentacion.pptx     # presentación
├── datos.xlsx
└── nido/                 # ← espacio wiki
    ├── introduccion.md
    ├── arquitectura/
    │   ├── vision-general.md
    │   └── base-de-datos.md
    └── guia-de-uso.md
```

Propiedades

- La wiki es el sistema de ficheros: cada página es un .md (Git, portable).
- Bidireccional: editar en disco actualiza la wiki y viceversa.
- Documentos anidados (nested docs) sin límite de nivel.
- Hereda las ACL de la carpeta: mismos usuarios y grupos.
- Varios Nidos por carpeta (ingeniería, negocio…).

Fast Access — carpetas en RAM (in-memory)

- ✦ Idea tomada de las bases de datos in-memory (como SAP HANA, visto en Bases de Datos): los datos viven en RAM y el disco hace de respaldo.
- ✦ El usuario, según permisos y cuota, marca carpeta/archivo para residir en RAM mientras el servidor esté encendido.
- ✦ Cada escritura va a un diario (WAL) + volcado atómico a disco → el disco es la fuente de verdad. Cero pérdida.
- ✦ Si la RAM se llena: desalojo LRU → se libera la copia en RAM, el dato sigue en disco.

Modo	Qué hace	Pérdida máxima
Durable (por defecto)	Diario + escritura a disco en cada operación	0
Equilibrado	Diario en RAM por lotes muy cortos	Casi 0
Volátil	Solo caché, para datos temporales	Lo no guardado

Adiciones: diagramas y visor HTML

Diagramas integrados (tipo draw.io)

- Crea y edita diagramas de flujo, arquitectura o UML dentro de Kortex.
- Se guardan como un archivo del proyecto, junto al resto de archivos.
- Enlazables desde una página de un Nido (como una imagen viva).

Editor de HTML/CSS con visor

- Editas el código y ves el resultado en vivo.
- Vista dividida: código a la izquierda, previsualización a la derecha.
- Autoguardado: cada guardado refresca el visor al instante.
- Modo presentar: el HTML ocupa toda la pantalla.

Fases del desarrollo aplicadas

Fase (Tema 2)	Aplicación en Kortex
Requisitos	Entrevistas con empresas piloto; este documento es el resultado.
Análisis	Casos de uso y modelo de datos (usuarios, grupos, carpetas, Nidos, páginas).
Diseño	Web, API REST, SMB/WebDAV, edición, búsqueda, MCP y caché RAM/WAL.
Implementación	Sprints de 2 semanas.
Pruebas	Unitarias, integración, permisos heredados, carga y durabilidad (corte de luz).
Documentación / Explotación / Mantenimiento	Guías, referencia de CLI y API MCP; 1.0 y evolutivos.

Casos de uso principales (UML)

- CU1 Publicar una página desde la CLI · Integrador
- CU2 Editar un documento ofimático en tiempo real · Editor
- CU3 Buscar documentación en archivos y Nidos · Editor / Lector
- CU4 Redactar y publicar un acta automáticamente · Agente IA (MCP)
- CU5 Gestionar usuarios y permisos (grupos/ACL) · Administrador
- CU6 Montar la carpeta como unidad de red · Editor
- CU7 Crear un Nido dentro de una carpeta · Editor
- CU8 Activar Fast Access en una carpeta · Editor
- CU9 Restaurar una versión anterior · Editor / Administrador
- CU10 Definir cuotas de RAM y política de durabilidad · Administrador

Conclusión

✦ Kortex parte de una toma de requisitos real, se analiza con casos de uso y se diseña antes de programar: ingeniería del software aplicada de principio a fin.

✦ Se desarrolla con Scrum y un ciclo en espiral, asignando roles concretos al equipo.

✦ Propuesta de valor: wiki + ofimática + almacén seguro + carpetas en RAM, bajo una sola carpeta y una sola identidad.

Kortex: una idea realista, planificada con la ingeniería del software del Tema 2.

Gracias

¿Preguntas?

Kortex — Actividad 2 · Entornos de Desarrollo